

**Comenzado el** viernes, 22 de mayo de 2026, 01:33**Estado** Finalizado**Finalizado en** viernes, 22 de mayo de 2026, 01:59**Tiempo empleado** 25 minutos 48 segundos**Calificación** 100,00 de 100,00**Pregunta 1** | Finalizado Se puntúa 1,34 sobre 1,34

Dado el siguiente código en java:

```
public class Principal {  
    public static void main(String[] args) {  
        int[] numeros = {10, 20, 30, 40};  
    }  
}
```

Si se desea mostrar en consola el contenido completo del arreglo en una sola línea con el siguiente formato [10, 20 30, 40] , ¿cuál de las siguientes opciones es la correcta?

- ☒ a. System.out.println(Arrays.toString(numeros));
- ☐ b. System.out.println(numeros.toString());
- ☐ c. System.out.println(numeros);
- ☐ d. for (int n : numeros)
System.out.print(n);



Pregunta 2 | Finalizado Se puntúa 1,34 sobre 1,34

Dado el siguiente código en lenguaje Java :

```
public class Array {  
    public static void main(String[] args) {  
        Libro libros[] = new Libro[3];  
  
        libros[0] = new Libro( titulo: "SQL");  
        libros[1] = new Libro( titulo: "Python");  
        libros[2] = new Libro( titulo: "C#");  
        libros[1] = new Libro( titulo: "HTML");  
        libros[0] = new Libro( titulo: "Java");  
  
        for (Libro l : libros) {  
            System.out.print(l.getTitulo() + " ");  
        }  
    }  
}
```

¿ Qué se muestra en la consola al ejecutar este programa ?

- ☐ a. SQL Python C# HTML Java
- ☐ b. HTML C# Java
- ☐ c. Java HTML SQL
- ☒ d. Java HTML C#



Pregunta 3 | Correcta Se puntúa 6,00 sobre 6,00

Objetivo: Trabajar con **clases abstractas**, **herencia**, **sobrescritura de métodos** y **polimorfismo** a partir del cálculo de consumo de distintos tipos de autos.

Se cuenta con:

- Una clase abstracta **Auto** que define el método abstracto:

```
calcularConsumo()
```

Este método devuelve el consumo en pesos del auto para un recorrido determinado.

- Tres clases que heredan de **Auto**:

- **AutoNafta**
- **AutoHibrido**
- **AutoElectrico**

- Cada una de estas clases sobrescribe el método **calcularConsumo()** de la siguiente manera:

- **AutoNafta**: el consumo se calcula multiplicando los kilómetros recorridos por el costo por kilómetro.
- **AutoHibrido**: el consumo se calcula multiplicando los kilómetros por el costo por kilómetro y luego aplicando un **30% de ahorro**.
- **AutoElectrico**: el consumo se calcula multiplicando los kilómetros por el costo por kilómetro y luego aplicando un **50% de ahorro**.

- Una clase **Mostrar** donde se debe implementar un método **static** llamado:

```
mostrarDatos
```

✓ **Tu tarea:** Implementar el método **mostrarDatos** para que reciba como parámetro una lista de objetos de tipo **Auto** y permita:

- Recorrer la lista.
- Indicar de qué tipo es cada auto.
- Calcular el consumo de cada uno.
- Mostrar al final la suma total de consumos en pesos.

Importante: El método debe trabajar con una lista de objetos de tipo **Auto**, aprovechando el polimorfismo para invocar **calcularConsumo()** según el tipo real de cada objeto.

Por ejemplo:

Prueba	Resultado
<pre>java.util.List<Auto> lista = new java.util.ArrayList<>(); lista.add(new AutoNafta(100, 12)); lista.add(new AutoHibrido(100, 12)); lista.add(new AutoElectrico(100, 12)); Mostrar.mostrarDatos(lista);</pre>	<pre>AutoNafta AutoHibrido AutoElectrico 2640.0</pre>
<pre>java.util.List<Auto> lista = new java.util.ArrayList<>(); lista.add(new AutoNafta(100, 10)); lista.add(new AutoHibrido(100, 10)); lista.add(new AutoElectrico(100, 10)); lista.add(new AutoNafta(50, 8)); lista.add(new AutoHibrido(50, 8)); lista.add(new AutoElectrico(50, 8)); Mostrar.mostrarDatos(lista);</pre>	<pre>AutoNafta AutoHibrido AutoElectrico AutoNafta AutoHibrido AutoElectrico 3080.0</pre>

Respuesta: (sistema de penalización: 0, 0, 0, 10, 20, 30, ... %)

Reiniciar respuesta

1 | // Clase abstracta Auto



```

2
3 abstract class Auto {
4     String tipo;
5
6     public Auto(String tipo) {
7         this.tipo = tipo;
8     }
9
10    public abstract double calcularConsumo();
11 }
12
13 // Clase AutoNafta
14
15 class AutoNafta extends Auto {
16     private double kilometros;
17     private double costoPorKm;
18
19     public AutoNafta(double kilometros, double costoPorKm) {
20         super("Nafta");
21         this.kilometros = kilometros;
22         this.costoPorKm = costoPorKm;
23     }
24
25     @Override
26     public double calcularConsumo() {
27         return kilometros * costoPorKm;
28     }
29 }
30
31 // Clase AutoHibrido
32
33 class AutoHibrido extends Auto {
34     private double kilometros;
35     private double costoPorKm;
36
37     public AutoHibrido(double kilometros, double costoPorKm) {
38         super("Hibrido");
39         this.kilometros = kilometros;
40         this.costoPorKm = costoPorKm;
41     }
42
43     @Override
44     public double calcularConsumo() {
45         return kilometros * costoPorKm * 0.70;
46     }
47 }
48
49 // Clase AutoElectrico
50
51 class AutoElectrico extends Auto {
52     private double kilometros;

```

	Prueba	Esperado	Conseguido	
✓	<pre> java.util.List<Auto> lista = new java.util.ArrayList<>(); lista.add(new AutoNafta(100, 12)); lista.add(new AutoHibrido(100, 12)); lista.add(new AutoElectrico(100, 12)); Mostrar.mostrarDatos(lista); </pre>	<pre> AutoNafta AutoHibrido AutoElectrico 2640.0 </pre>	<pre> AutoNafta AutoHibrido AutoElectrico 2640.0 </pre>	✓
✓	<pre> java.util.List<Auto> lista = new java.util.ArrayList<>(); lista.add(new AutoNafta(100, 10)); lista.add(new AutoHibrido(100, 10)); lista.add(new AutoElectrico(100, 10)); lista.add(new AutoNafta(50, 8)); lista.add(new AutoHibrido(50, 8)); lista.add(new AutoElectrico(50, 8)); Mostrar.mostrarDatos(lista); </pre>	<pre> AutoNafta AutoHibrido AutoElectrico AutoNafta AutoHibrido AutoElectrico 3080.0 </pre>	<pre> AutoNafta AutoHibrido AutoElectrico AutoNafta AutoHibrido AutoElectrico 3080.0 </pre>	✓



	Prueba	Esperado	Conseguido	
✓	<pre>java.util.List<Auto> lista = new java.util.ArrayList<>(); lista.add(new AutoNafta(0, 12)); lista.add(new AutoHibrido(100, 12)); lista.add(new AutoElectrico(100, 12)); lista.add(new AutoNafta(50, 12)); Mostrar.mostrarDatos(lista);</pre>	AutoNafta AutoHibrido AutoElectrico AutoNafta 2040.0	AutoNafta AutoHibrido AutoElectrico AutoNafta 2040.0	✓
✓	<pre>java.util.List<Auto> lista = new java.util.ArrayList<>(); lista.add(new AutoNafta(100, 0)); lista.add(new AutoHibrido(100, 0)); lista.add(new AutoElectrico(100, 0)); lista.add(new AutoNafta(80, 5)); Mostrar.mostrarDatos(lista);</pre>	AutoNafta AutoHibrido AutoElectrico AutoNafta 400.0	AutoNafta AutoHibrido AutoElectrico AutoNafta 400.0	✓
✓	<pre>java.util.List<Auto> lista = new java.util.ArrayList<>(); lista.add(new AutoElectrico(150, 6)); lista.add(new AutoNafta(150, 6)); lista.add(new AutoHibrido(150, 6)); lista.add(new AutoElectrico(60, 9)); lista.add(new AutoNafta(60, 9)); lista.add(new AutoHibrido(60, 9)); Mostrar.mostrarDatos(lista);</pre>	AutoElectrico AutoNafta AutoHibrido AutoElectrico AutoNafta AutoHibrido 3168.0	AutoElectrico AutoNafta AutoHibrido AutoElectrico AutoNafta AutoHibrido 3168.0	✓
✓	<pre>java.util.List<Auto> lista = new java.util.ArrayList<>(); lista.add(new AutoNafta(75, 10)); lista.add(new AutoHibrido(75, 2)); lista.add(new AutoElectrico(75, 6)); lista.add(new AutoNafta(20, 4)); Mostrar.mostrarDatos(lista);</pre>	AutoNafta AutoHibrido AutoElectrico AutoNafta 1160.0	AutoNafta AutoHibrido AutoElectrico AutoNafta 1160.0	✓
✓	<pre>java.util.List<Auto> lista = new java.util.ArrayList<>(); lista.add(new AutoNafta(100, 7)); lista.add(new AutoNafta(120, 7)); lista.add(new AutoNafta(80, 7)); lista.add(new AutoHibrido(100, 7)); lista.add(new AutoElectrico(100, 7)); Mostrar.mostrarDatos(lista);</pre>	AutoNafta AutoNafta AutoNafta AutoHibrido AutoElectrico 2940.0	AutoNafta AutoNafta AutoNafta AutoHibrido AutoElectrico 2940.0	✓

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 6,00/6,00.



Pregunta 4 | Finalizado Se puntúa 1,33 sobre 1,33

Dada el siguiente código en lenguaje Java :

```
public class PruebaString {  
    public static void main(String[] args) {  
        String x = "Java";  
        String y = "Java";  
        String z = new String( original: "Java");  
  
        System.out.print(x == y);  
        System.out.print(" ");  
        System.out.print(x == z);  
        System.out.print(" ");  
        System.out.print(x.equals(z));  
    }  
}
```

¿ Qué se muestra en la consola al ejecutar este programa ?

- ☐ a. true true true
- ☐ b. true false false
- ☐ c. false true true
- ☒ d. true false true

Pregunta 5 | Finalizado Se puntúa 1,33 sobre 1,33

¿Cuál de las siguientes opciones representa correctamente la definición de un método sin retorno?

- ☐ a. public suma() { }
- ☒ b. public void suma() { }
- ☐ c. void public suma() { }
- ☐ d. public int suma(void) { }

Pregunta 6 | Finalizado Se puntúa 1,33 sobre 1,33

Dadas las siguientes afirmaciones sobre el manejo de excepciones en Java:

- 1 - El bloque try contiene el código que puede producir una excepción.
- 2 - El bloque catch se ejecuta siempre, ocurra o no una excepción.
- 3 - El bloque finally se ejecuta siempre haya o no excepción.
- 4 - La palabra throw se usa para lanzar una excepción.
- 5 - La palabra throws se usa dentro del bloque catch.
- 6 - Un try no puede existir sin al menos un catch o un finally.

¿Cuál de las siguientes opciones contiene solamente afirmaciones correctas?

- ☐ a. 1, 2, 3 y 5
- ☐ b. 1, 4, 5 y 6
- ☐ c. 2, 3, 4 y 5
- ☒ d. 1, 3, 4 y 6



Pregunta 7 | Finalizado Se puntúa 1,34 sobre 1,34

Dado el siguiente código en lenguaje Java en donde participan 2 clases :

```
public class Mostrar { no usages
}
    public void mostrar(int a, int b) { no usages
        System.out.println("Metodo int: " + (a + b));
    }

    public void mostrar(double a, double b) { no usages
        System.out.println("Metodo double: " + (a + b));
    }

    public void mostrar(byte a, byte b) { no usages
        System.out.println("Metodo byte: " + (a + b));
    }
}

public class TestMostrar {
}
    public static void main(String[] args) {
        Mostrar m = new Mostrar();
        m.mostrar((short)5, (short)7);
    }
}
```

¿ Qué se muestra en la consola al ejecutar el programa TestMostrar ?

- ☒ a. Metodo int: 12
- ☐ b. Da error de compilación.
- ☐ c. Metodo double: 12.0
- ☐ d. Metodo byte: 12

Pregunta 8 | Finalizado Se puntúa 1,33 sobre 1,33

Dada la siguiente estructura de código en Java:

```
// Bloque 1
try {
    // Bloque 2
} catch (NullPointerException ex) {
    // Bloque 3
} catch (Exception e) {
    // Bloque 4
} finally {
    // Bloque 5
}
// Bloque 6
```

Si se ejecuta este código y se produce una excepción en el Bloque 2 del tipo NumberFormatException, ¿ cuál es el flujo correcto ?

- ☐ a. Bloque 1 -> Bloque 2 -> Bloque 4 -> Bloque 6
- ☒ b. Bloque 1 -> Bloque 2 -> Bloque 4 -> Bloque 5 -> Bloque 6
- ☐ c. Bloque 1 -> Bloque 2 -> Bloque 3 -> Bloque 5 -> Bloque 6
- ☐ d. Bloque 2 -> Bloque 4 -> Bloque 6



Pregunta 9 | Correcta Se puntúa 6,00 sobre 6,00

Completar la función `padovan(int n)`.

La sucesión de Padovan se define de la siguiente manera:

- $P(0) = 1$
- $P(1) = 1$
- $P(2) = 1$

A partir de esos valores iniciales, cada término se calcula como la suma de los términos ubicados dos y tres posiciones antes:

$$P(n) = P(n-2) + P(n-3)$$

Los primeros términos de la sucesión de Padovan son:

1 1 1 2 2 3 4 5 7 9

El método debe devolver un array con los primeros n términos de la sucesión.

Por ejemplo:

- si $n = 10$, debe devolver un array con los valores: 1 1 1 2 2 3 4 5 7 9

Aclaraciones:

- si $n \leq 0$, el método debe devolver un array vacío.
- si $n = 1$, debe devolver: 1
- si $n = 2$, debe devolver: 1 1
- si $n = 3$, debe devolver: 1 1 1

La firma del método es la siguiente:

```
public int[] padovan(int n)
```

Importante:

- No debes incluir el método `main`.
- No debes realizar pruebas manuales ni llamadas al método.
- Solo debes completar la lógica del método, ya que las pruebas serán realizadas automáticamente por CodeRunner.

Por ejemplo:

Prueba	Resultado
<code>System.out.println(java.util.Arrays.toString(padovan(0)));</code>	<code>[]</code>
<code>System.out.println(java.util.Arrays.toString(padovan(10)));</code>	<code>[1, 1, 1, 2, 2, 3, 4, 5, 7, 9]</code>

Respuesta: (sistema de penalización: 0, 0, 0, 10, 20, ... %)

Reiniciar respuesta

```

1 public int[] padovan(int n) {
2
3     if (n <= 0) {
4         return new int[0];
5     }
6
7     int[] serie = new int[n];
8
9     for (int i = 0; i < n; i++) {
10        if (i <= 2) {
11            serie[i] = 1;
12        } else {
13            serie[i] = serie[i - 2] + serie[i - 3];
14        }
15    }
16
17    return serie;
18 }
```



	Prueba	Esperado	Conseguido	
✓	System.out.println(java.util.Arrays.toString(padovan(0)));	[]	[]	✓
✓	System.out.println(java.util.Arrays.toString(padovan(10)));	[1, 1, 1, 2, 2, 3, 4, 5, 7, 9]	[1, 1, 1, 2, 2, 3, 4, 5, 7, 9]	✓
✓	System.out.println(java.util.Arrays.toString(padovan(-3)));	[]	[]	✓
✓	System.out.println(java.util.Arrays.toString(padovan(1)));	[1]	[1]	✓
✓	System.out.println(java.util.Arrays.toString(padovan(4)));	[1, 1, 1, 2]	[1, 1, 1, 2]	✓
✓	System.out.println(java.util.Arrays.toString(padovan(15)));	[1, 1, 1, 2, 2, 3, 4, 5, 7, 9, 12, 16, 21, 28, 37]	[1, 1, 1, 2, 2, 3, 4, 5, 7, 9, 12, 16, 21, 28, 37]	✓
✓	System.out.println(java.util.Arrays.toString(padovan(20)));	[1, 1, 1, 2, 2, 3, 4, 5, 7, 9, 12, 16, 21, 28, 37, 49, 65, 86, 114, 151]	[1, 1, 1, 2, 2, 3, 4, 5, 7, 9, 12, 16, 21, 28, 37, 49, 65, 86, 114, 151]	✓

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 6,00/6,00.

Pregunta 10 | Finalizado Se puntúa 1,33 sobre 1,33

Dado el siguiente código en Java:

```
public class PruebaExcepcion {
    public static void main(String[] args) {
        try {
            int[] numeros = {10, 20, 30};

            try {
                System.out.println(numeros[5]);
            } catch (ArithmeticException e) {
                System.out.println("Error aritmético");
            }

            System.out.println("Fin del bloque interno");

        } catch (NullPointerException e) {
            System.out.println("Error de referencia nula");
        }

        System.out.println("Fin del programa");
    }
}
```

¿ Qué sucede al ejecutar este programa ?

- ☐ a. Se muestra Fin del bloque interno y luego Fin del programa
- ☐ b. Se muestra Error aritmético y luego Fin del programa
- ☒ c. Se produce una excepción no controlada y el programa se detiene
- ☐ d. Se muestra Error de referencia nula y luego Fin del programa



Pregunta 11 | Finalizado Se puntúa 1,33 sobre 1,33

¿Cuál de las siguientes afirmaciones es correcta sobre una interfaz en Java?

- ☐ a. Los métodos de una interfaz siempre deben ser private.
- ☒ b. Una clase que implementa una interfaz debe implementar sus métodos abstractos.
- ☐ c. Una interfaz puede ser instanciada directamente usando new.
- ☐ d. Una interfaz no puede contener métodos.

Pregunta 12 | Finalizado Se puntúa 1,34 sobre 1,34

Dado el siguiente código en Java :

```
public interface Vehiculo {  
    void acelerar();  
    void frenar();  
  
    default void describir() {  
        System.out.println("Yo soy un vehículo.");  
    }  
}
```

Y tenemos las siguientes afirmaciones:

- 1 - Esta interfaz esta definida correctamente
- 2- Se puede crear una instancia de Vehiculo.
- 3- Esta interfaz no esta definida correctamente, no puede tener un método implementado.
- 4 - Para implementar esta interfaz desde otra clase debemos usar la palabra reservada implements.
- 5- Para implementar esta interfaz desde otra clase debemos usar la palabra reservada extends.
- 6- Si implementamos esta interfaz debemos implementar todos sus métodos abstractos.

¿ Cuales de las siguientes afirmaciones son verdaderas ?

- ☐ a. 4, 6
- ☐ b. 2, 3, 5.
- ☐ c. 2, 3, 5, 6.
- ☒ d. 1, 4 , 6



Pregunta 13 | Finalizado Se puntúa 1,34 sobre 1,34

Dado el siguiente código en lenguaje Java:

```
public class PrincipalSet {  
    public static void main(String[] args) {  
        Set<Integer> numeros = new LinkedHashSet<>();  
  
        numeros.add(10);  
        numeros.add(20);  
        numeros.add(10);  
        numeros.add(30);  
        numeros.add(20);  
  
        System.out.println(numeros);  
    }  
}
```

¿ Qué se muestra en la consola cuando se ejecuta este programa ?

- ☐ a. [10, 20, 20, 30]
- ☐ b. [30, 10, 20]
- ☐ c. [10, 20, 10, 30, 20]
- ☒ d. [10, 20, 30]

Pregunta 14 | Finalizado Se puntúa 1,34 sobre 1,34

Dado el siguiente código en lenguaje Java :

```
public class PrincipalCola {  
    public static void main(String[] args) {  
        Queue<String> nombres = new LinkedList<>();  
        nombres.offer("Antonio");  
        nombres.offer("Carmen");  
        nombres.offer("Mariana");  
        nombres.offer("Dante");  
  
        nombres.poll();  
        nombres.poll();  
  
        nombres.offer("Oscar");  
  
        System.out.println(nombres);  
    }  
}
```

¿ Qué se imprime en la consola al ejecutar este programa ?

- ☒ a. [Mariana, Dante, Oscar]
- ☐ b. [Antonio, Carmen, Mariana, Dante, Oscar]
- ☐ c. [Antonio, Carmen, Oscar]
- ☐ d. [Dante, Antonio, Oscar]



Pregunta 15 | Finalizado Se puntúa 1,33 sobre 1,33

Si se ejecuta :

```
String texto = "Programacion";  
System.out.println(texto.substring(0, 4));
```

¿ Qué se muestra ?

- ☒ a. Prog
- ☐ b. Programacion
- ☐ c. gram
- ☐ d. Program

Pregunta 16 | Finalizado Se puntúa 1,33 sobre 1,33

¿Cuál de las siguientes afirmaciones describe mejor una clase abstracta en Java?

- ☐ a. Una clase que tiene que tener todos sus métodos abstractos.
- ☐ b. Una clase que no puede tener variables de instancia.
- ☒ c. Una clase que no se puede instanciar directamente.
- ☐ d. Una clase que no puede heredar de otras clases.



Pregunta 17 | Correcta Se puntúa 6,00 sobre 6,00

Escribe una clase en Java que modele una clase llamada OperacionesConArray, que en ella tengas tres métodos:

- **sumaElementosMatriz():** Método que recibe una matriz de dos dimensiones y calcula la suma todos los elementos de la matriz, devolviendo un entero.
- **sumarMatrices():** Método que recibe dos matrices de dos dimensiones y suma ambas matrices devolviendo la matriz resultante.
- **imprimirMatriz():** Método que recibe un matriz dos dimensiones e imprime la misma en un formato de matriz, el método debe ser de tipo void.

Importante, todas las matrices son de enteros (int).

Recuerda que luego nosotros nos encargamos de probarlos en el método **main**, Por lo tanto no debes desarrollarlo.

Por ejemplo:

Prueba	Resultado
<pre>OperacionesConMatrices operaciones = new OperacionesConMatrices(); int[][] matrizEnteros = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9} }; operaciones.imprimirMatriz(matrizEnteros); System.out.println("La suma de los elementos de la matriz es: " + operaciones.sumaElementosMatriz(matrizEnteros));</pre>	<pre>1 2 3 4 5 6 7 8 9 La suma de los elementos de la matriz es: 45</pre>
<pre>OperacionesConMatrices operaciones = new OperacionesConMatrices(); int[][] matriz1 = { {1, 2}, {3, 4} }; int[][] matriz2 = { {5, 6}, {7, 8} }; System.out.println("Matriz 1:"); operaciones.imprimirMatriz(matriz1); System.out.println("Matriz 2:"); operaciones.imprimirMatriz(matriz2); int[][] resultado = operaciones.sumarMatrices(matriz1, matriz2); System.out.println("Resultado de la suma de matrices:"); operaciones.imprimirMatriz(resultado);</pre>	<pre>Matriz 1: 1 2 3 4 Matriz 2: 5 6 7 8 Resultado de la suma de matrices: 6 8 10 12</pre>

Respuesta: (sistema de penalización: 0, 0, 0, 10, 20, ... %)

Reiniciar respuesta

```

1 public class OperacionesConMatrices {
2
3     // Método para sumar todos los elementos de una matriz
4     public int sumaElementosMatriz(int[][] matriz) {
5         int suma = 0;
6
7         for (int fila = 0; fila < matriz.length; fila++) {
8             for (int columna = 0; columna < matriz[fila].length; columna++) {
9                 suma += matriz[fila][columna];
10            }
11        }
12
13        return suma;
14    }
15
16
17    // Método para imprimir una matriz
18    public void imprimirMatriz(int[][] matriz) {
19        for (int fila = 0; fila < matriz.length; fila++) {
20            for (int columna = 0; columna < matriz[fila].length; columna++) {
21                System.out.print(matriz[fila][columna] + "\t");
22            }
23        }
24    }
25 }
```



```

23         System.out.println();
24     }
25 }
26
27
28 // Método para sumar dos matrices
29 public int[][] sumarMatrices(int[][] matriz1, int[][] matriz2) {
30     int filas = matriz1.length;
31     int columnas = matriz1[0].length;
32
33     int[][] resultado = new int[filas][columnas];
34
35     for (int fila = 0; fila < filas; fila++) {
36         for (int columna = 0; columna < columnas; columna++) {
37             resultado[fila][columna] = matriz1[fila][columna] + matriz2[fila][columna];
38         }
39     }
40
41     return resultado;
42 }
43 }

```

	Prueba	Esperado	Conseguido	
✓	OperacionesConMatrices operaciones = new OperacionesConMatrices(); int[][] matrizEnteros = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9} }; operaciones.imprimirMatriz(matrizEnteros); System.out.println("La suma de los elementos de la matriz es: " + operaciones.sumaElementosMatriz(matrizEnteros));	1\t2\t3\t 4\t5\t6\t 7\t8\t9\t La suma de los elementos de la matriz es: 45	1\t2\t3\t 4\t5\t6\t 7\t8\t9\t La suma de los elementos de la matriz es: 45	✓
✓	OperacionesConMatrices operaciones = new OperacionesConMatrices(); int[][] matrizNegativos = { {-1, -2, -3}, {-4, -5, -6}, {-7, -8, -9} }; operaciones.imprimirMatriz(matrizNegativos); System.out.println("La suma de los elementos de la matriz es: " + operaciones.sumaElementosMatriz(matrizNegativos));	-1\t-2\t-3\t -4\t-5\t-6\t -7\t-8\t-9\t La suma de los elementos de la matriz es: -45	-1\t-2\t-3\t -4\t-5\t-6\t -7\t-8\t-9\t La suma de los elementos de la matriz es: -45	✓
✓	OperacionesConMatrices operaciones = new OperacionesConMatrices(); int[][] matrizVacía = {}; operaciones.imprimirMatriz(matrizVacía); System.out.println("La suma de los elementos de la matriz es: " + operaciones.sumaElementosMatriz(matrizVacía));	La suma de los elementos de la matriz es: 0	La suma de los elementos de la matriz es: 0	✓
✓	OperacionesConMatrices operaciones = new OperacionesConMatrices(); int[][] matrizUnElemento = {{5}}; operaciones.imprimirMatriz(matrizUnElemento); System.out.println("La suma de los elementos de la matriz es: " + operaciones.sumaElementosMatriz(matrizUnElemento));	5\t La suma de los elementos de la matriz es: 5	5\t La suma de los elementos de la matriz es: 5	✓
✓	OperacionesConMatrices operaciones = new OperacionesConMatrices(); int[][] matrizMixta = { {-1, 2, -3}, {4, -5, 6}, {-7, 8, -9} }; operaciones.imprimirMatriz(matrizMixta); System.out.println("La suma de los elementos de la matriz es: " + operaciones.sumaElementosMatriz(matrizMixta));	-1\t2\t-3\t 4\t-5\t6\t -7\t8\t-9\t La suma de los elementos de la matriz es: -5	-1\t2\t-3\t 4\t-5\t6\t -7\t8\t-9\t La suma de los elementos de la matriz es: -5	✓



	Prueba	Esperado	Conseguido	
✓	<pre>OperacionesConMatrices operaciones = new OperacionesConMatrices(); int[][] matriz1 = { {1, 2}, {3, 4} }; int[][] matriz2 = { {5, 6}, {7, 8} }; System.out.println("Matriz 1:"); operaciones.imprimirMatriz(matriz1); System.out.println("Matriz 2:"); operaciones.imprimirMatriz(matriz2); int[][] resultado = operaciones.sumarMatrices(matriz1, matriz2); System.out.println("Resultado de la suma de matrices:"); operaciones.imprimirMatriz(resultado);</pre>	<pre>Matriz 1: 1\t2\t 3\t4\t Matriz 2: 5\t6\t 7\t8\t Resultado de la suma de matrices: 6\t8\t 10\t12\t</pre>	<pre>Matriz 1: 1\t2\t 3\t4\t Matriz 2: 5\t6\t 7\t8\t Resultado de la suma de matrices: 6\t8\t 10\t12\t</pre>	✓
✓	<pre>OperacionesConMatrices operaciones = new OperacionesConMatrices(); int[][] matriz1 = { {-1, -2}, {-3, -4} }; int[][] matriz2 = { {-5, -6}, {-7, -8} }; System.out.println("Matriz 1:"); operaciones.imprimirMatriz(matriz1); System.out.println("Matriz 2:"); operaciones.imprimirMatriz(matriz2); int[][] resultado = operaciones.sumarMatrices(matriz1, matriz2); System.out.println("Resultado de la suma de matrices:"); operaciones.imprimirMatriz(resultado);</pre>	<pre>Matriz 1: -1\t-2\t -3\t-4\t Matriz 2: -5\t-6\t -7\t-8\t Resultado de la suma de matrices: -6\t-8\t -10\t-12\t</pre>	<pre>Matriz 1: -1\t-2\t -3\t-4\t Matriz 2: -5\t-6\t -7\t-8\t Resultado de la suma de matrices: -6\t-8\t -10\t-12\t</pre>	✓
✓	<pre>OperacionesConMatrices operaciones = new OperacionesConMatrices(); int[][] matriz1 = {{5}}; int[][] matriz2 = {{10}}; System.out.println("Matriz 1:"); operaciones.imprimirMatriz(matriz1); System.out.println("Matriz 2:"); operaciones.imprimirMatriz(matriz2); int[][] resultado = operaciones.sumarMatrices(matriz1, matriz2); System.out.println("Resultado de la suma de matrices:"); operaciones.imprimirMatriz(resultado);</pre>	<pre>Matriz 1: 5\t Matriz 2: 10\t Resultado de la suma de matrices: 15\t</pre>	<pre>Matriz 1: 5\t Matriz 2: 10\t Resultado de la suma de matrices: 15\t</pre>	✓

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 6,00/6,00.



Pregunta 18 | Finalizado Se puntúa 1,33 sobre 1,33

¿Cuál de los siguientes casos representa una sobrecarga válida?

- ☐ a. `sumar(int a, int b)` y `restar(int a, int b)`
- ☐ b. `sumar(int a, int b)` y `sumar(int g, int h)`
- ☒ c. `sumar(int a, int b)` y `sumar(double a, double b)`
- ☐ d. `sumar(int a, int b)` y `sumar(int x, int y)`

Pregunta 19 | Finalizado Se puntúa 1,33 sobre 1,33

Indica cuáles de las siguientes afirmaciones NO son correctas:

- 1 - `catch` puede existir sin `try`.
- 2 - `finally` puede usarse junto con `try`.
- 3 - `throw` se usa para lanzar una excepción.
- 4 - `throws` se usa en la firma del método.
- 5 - El bloque `finally` se ejecuta únicamente si ocurre excepción.
- 6 - Un `catch` puede capturar una excepción del tipo indicado.

¿Qué opción contiene todas las afirmaciones incorrectas?

- ☐ a. 1, 3 y 4
- ☐ b. 2, 4 y 6
- ☒ c. 1 y 5
- ☐ d. 3, 5 y 6



Pregunta 20 | Finalizado Se puntúa 1,33 sobre 1,33

Dado el siguiente código en lenguaje Java :

```
public class Operador{  
    public static void main(String[] args) {  
        int a = 10, b = 4, c = 7;  
        a--;  
        b += 3;  
        c--;  
  
        for (int i = 1; i <= 2; i++) {  
            a -= 2;  
            b++;  
        }  
  
        boolean oper = a + b > 10 || c < 6;  
        int suma = a + b + c;  
        String mensaje = (suma % 2 == 0) ? "PAR" : "IMPAR";  
  
        System.out.println(oper);  
        System.out.println(suma);  
        System.out.println(mensaje);  
        System.out.println("variable a es: "+a);  
    }  
}
```

Luego de ejecutar este código, ¿cuál de las siguientes afirmaciones es falsa ?

- ☐ a. El valor final de la variable a es 5.
- ☐ b. El valor que se imprime de suma es 20.
- ☐ c. El valor que se imprime de oper es true.
- ☒ d. El valor que se imprime de mensaje es "IMPAR"

Pregunta 21 | Finalizado Se puntúa 1,33 sobre 1,33

¿Cuál de las siguientes formas crea correctamente un objeto LocalDate ?

- ☐ a. LocalDate fecha = LocalDate.create(2026, 04, 30)
- ☒ b. LocalDate fecha = LocalDate.of(2026, 04, 30)
- ☐ c. LocalDate fecha = Date.of(2026, 04, 30)
- ☐ d. LocalDate fecha = new LocalDate(2026, 04, 30)

Pregunta 22 | Finalizado Se puntúa 1,33 sobre 1,33

¿Cuál de las siguientes afirmaciones sobre nombres de variables en Java es correcta?

- ☒ a. No pueden usar palabras reservadas del lenguaje
- ☐ b. Pueden comenzar con un número
- ☐ c. Deben escribirse siempre en mayúsculas
- ☐ d. Pueden contener guiones medios -



Pregunta 23 | Finalizado Se puntúa 1,33 sobre 1,33

Dado el siguiente código en lenguaje Java:

```
import java.util.HashMap;
import java.util.Map;

public class PruebaMap {
    public static void main(String[] args) {
        Map<String, Integer> stock = new HashMap<>();

        stock.put("Lapiz", 10);
        stock.put("Cuaderno", 5);
        stock.put("Goma", 8);

        stock.put("Lapiz", stock.get("Lapiz") + 5);
        stock.remove("Cuaderno");
        stock.put("Regla", 7);
        stock.put("Goma", stock.get("Goma") - 3);

        System.out.println(stock);
    }
}
```

¿Cuál de las siguientes afirmaciones es correcta respecto al estado final del Map?

- ☐ a. El Map contiene 4 elementos y "Goma" queda con valor 8
- ☐ b. El Map contiene 2 elementos y "Regla" no fue agregada
- ☐ c. El Map contiene 3 elementos, pero "Lapiz" queda con valor 10
- ☒ d. El Map contiene 3 elementos, "Lapiz" queda con valor 15 y "Cuaderno" ya no está

Pregunta 24 | Finalizado Se puntúa 1,33 sobre 1,33

¿Cuál es la principal diferencia entre una función y un procedimiento en Java?

- ☐ a. La función no puede ser public
- ☐ b. El procedimiento solo puede ser static
- ☒ c. La función devuelve un valor y el procedimiento no
- ☐ d. La función siempre recibe parámetros y el procedimiento no



Pregunta 25 | Correcta Se puntúa 6,00 sobre 6,00

Objetivo: Recorrer matrices bidimensionales (incluyendo matrices escalonadas) y calcular la suma de los elementos por fila.

Se cuenta con:

- Un método a implementar llamado:
`mostrarSumaFilas(int[][] matriz)`
- Una matriz bidimensional de números enteros, que puede ser **escalonada** (filas con distinta cantidad de elementos).

✓ **Tu tarea:** Implementar el método `mostrarSumaFilas` para que:

- Recorra la matriz fila por fila.
- Calcule la suma de los elementos de cada fila.
- Muestre el resultado respetando exactamente el siguiente formato:

Fila 1: xxxx

Fila 2: xxxx

Fila 3: xxxx

Donde **xxxx** representa la suma de los elementos de la fila correspondiente.

- Cada fila debe mostrarse en una línea separada.

Consideraciones:

- La numeración de las filas comienza en 1.
- Si una fila no tiene elementos, su suma es 0.
- No se debe implementar el método `main`.
- No se deben pedir datos por teclado.
- Se debe respetar exactamente el formato de salida solicitado.

Por ejemplo:

Prueba	Resultado
<pre>int[][] matriz = { {2, 4, 6}, {1, 3}, {5, 5, 5, 5} }; mostrarSumaFilas(matriz);</pre>	<pre>Fila 1: 12 Fila 2: 4 Fila 3: 20</pre>
<pre>int[][] matriz = { {1, 2, 3}, {4, 5}, {6} }; mostrarSumaFilas(matriz);</pre>	<pre>Fila 1: 6 Fila 2: 9 Fila 3: 6</pre>

Respuesta: (sistema de penalización: 0, 0, 0, 10, 20, ... %)

Reiniciar respuesta

```
1 public void mostrarSumaFilas(int[][] matriz) {
2     for (int fila = 0; fila < matriz.length; fila++) {
3         int suma = 0;
4
5         for (int columna = 0; columna < matriz[fila].length; columna++) {
6             suma += matriz[fila][columna];
7         }
8
9         System.out.println("Fila " + (fila + 1) + ": " + suma);
10    }
11 }
```



	Prueba	Esperado	Conseguido	
✓	<pre>int[][] matriz = { {2, 4, 6}, {1, 3}, {5, 5, 5, 5} }; mostrarSumaFilas(matriz);</pre>	Filas 1: 12 Filas 2: 4 Filas 3: 20	Filas 1: 12 Filas 2: 4 Filas 3: 20	✓
✓	<pre>int[][] matriz = { {1, 2, 3}, {4, 5}, {6} }; mostrarSumaFilas(matriz);</pre>	Filas 1: 6 Filas 2: 9 Filas 3: 6	Filas 1: 6 Filas 2: 9 Filas 3: 6	✓
✓	<pre>int[][] matriz = { {1, 2}, {}, {3, 4, 5} }; mostrarSumaFilas(matriz);</pre>	Filas 1: 3 Filas 2: 0 Filas 3: 12	Filas 1: 3 Filas 2: 0 Filas 3: 12	✓
✓	<pre>int[][] matriz = { {100, 200, 300}, {50}, {25, 25, 25, 25} }; mostrarSumaFilas(matriz);</pre>	Filas 1: 600 Filas 2: 50 Filas 3: 100	Filas 1: 600 Filas 2: 50 Filas 3: 100	✓
✓	<pre>int[][] matriz = { {12}, {}, {-5, 5}, {8, 1, 1} }; mostrarSumaFilas(matriz);</pre>	Filas 1: 12 Filas 2: 0 Filas 3: 0 Filas 4: 10	Filas 1: 12 Filas 2: 0 Filas 3: 0 Filas 4: 10	✓

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 6,00/6,00.



Pregunta 26 | Correcta Se puntúa 6,00 sobre 6,00

Objetivo: Implementar un CRUD sobre un **HashMap** usando una interfaz.

Se cuenta con:

- Una clase **Reserva** con los atributos:
 - **codigo** (int)
 - **titular** (String)
 - **noches** (int)
 - **precioPorNoche** (double)
- Una interfaz **ICrudReservas** que define operaciones CRUD sobre reservas.
- Una clase **CrudReservasImpl** que implementa la interfaz utilizando un:


```
java.util.HashMap<Integer, Reserva>
```

✓ **Tu tarea:** Completar los métodos marcados con comentarios en **CrudReservasImpl** para que el sistema permita:

- **Agregar** una reserva.
- **Modificar** una reserva existente (si no existe, imprimir mensaje).
- **Eliminar** una reserva existente (si no existe, imprimir mensaje).
- **Listar** todas las reservas mostrando: **Clave: X valor: ...**

Importante: No se deben usar **import**, usar **java.util.HashMap** y **java.util.Set** con nombre completo si hiciera falta.

Por ejemplo:

Prueba	Resultado
<pre>// ===== TEST 1: agregar + listar ===== ICrudReservas gestor = new CrudReservasImpl(); Reserva r1 = new Reserva(1, "Juan", 2, 100); Reserva r2 = new Reserva(2, "Ana", 5, 80); Reserva r3 = new Reserva(3, "Luis", 1, 150); gestor.agregar(r1); gestor.agregar(r2); gestor.agregar(r3); gestor.listar();</pre>	<pre>Clave: 1 valor: Reserva{codigo=1, titular='Juan', noches=2, precioPorNoche=100.0, total=200.0} Clave: 2 valor: Reserva{codigo=2, titular='Ana', noches=5, precioPorNoche=80.0, total=400.0} Clave: 3 valor: Reserva{codigo=3, titular='Luis', noches=1, precioPorNoche=150.0, total=150.0}</pre>

Respuesta: (sistema de penalización: 0, 0, 0, 10, 20, 30, ... %)

Reiniciar respuesta

```
1 // Clase Reserva
2 class Reserva {
3     private int codigo;
4     private String titular;
5     private int noches;
6     private double precioPorNoche;
7
8     public Reserva(int codigo, String titular, int noches, double precioPorNoche) {
9         this.codigo = codigo;
10        this.titular = titular;
11        this.noches = noches;
12        this.precioPorNoche = precioPorNoche;
13    }
14
15    public int getCodigo() { return codigo; }
```



```

15 public int getCodigo() { return codigo; }
16 public void setCodigo(int codigo) { this.codigo = codigo; }
17
18 public String getTitular() { return titular; }
19 public void setTitular(String titular) { this.titular = titular; }
20
21 public int getNoches() { return noches; }
22 public void setNoches(int noches) { this.noches = noches; }
23
24 public double getPrecioPorNoche() { return precioPorNoche; }
25 public void setPrecioPorNoche(double precioPorNoche) { this.precioPorNoche = precioPorNoche; }
26
27 public double total() {
28     return noches * precioPorNoche;
29 }
30
31 @Override
32 public String toString() {
33     return "Reserva{" +
34         "codigo=" + codigo +
35         ", titular='" + titular + '\'' +
36         ", noches=" + noches +
37         ", precioPorNoche=" + precioPorNoche +
38         ", total=" + total() +
39         '}';
40 }
41 }
42
43 // Interfaz CRUD
44 interface ICrudReservas {
45     void agregar(Reserva r);
46     void modificar(Reserva r);
47     void eliminar(int codigo);
48     void listar();
49 }
50
51 // Implementación completa
52 class CrudReservasImpl implements ICrudReservas {

```

	Prueba	Esperado	Conseguido	
✓	<pre>// ===== TEST 1: agregar + listar ===== ICrudReservas gestor = new CrudReservasImpl(); Reserva r1 = new Reserva(1, "Juan", 2, 100); Reserva r2 = new Reserva(2, "Ana", 5, 80); Reserva r3 = new Reserva(3, "Luis", 1, 150); gestor.agregar(r1); gestor.agregar(r2); gestor.agregar(r3); gestor.listar();</pre>	<pre>Clave: 1 valor: Reserva{codigo=1, titular='Juan', noches=2, precioPorNoche=100.0, total=200.0} Clave: 2 valor: Reserva{codigo=2, titular='Ana', noches=5, precioPorNoche=80.0, total=400.0} Clave: 3 valor: Reserva{codigo=3, titular='Luis', noches=1, precioPorNoche=150.0, total=150.0}</pre>	<pre>Clave: 1 valor: Reserva{codigo=1, titular='Juan', noches=2, precioPorNoche=100.0, total=200.0} Clave: 2 valor: Reserva{codigo=2, titular='Ana', noches=5, precioPorNoche=80.0, total=400.0} Clave: 3 valor: Reserva{codigo=3, titular='Luis', noches=1, precioPorNoche=150.0, total=150.0}</pre>	✓



	Prueba	Esperado	Conseguido	
✓	<pre>ICrudReservas gestor = new CrudReservasImpl(); Reserva r1 = new Reserva(1, "Juan", 2, 100); Reserva r2 = new Reserva(2, "Ana", 5, 80); Reserva r3 = new Reserva(3, "Luis", 1, 150); gestor.agregar(r1); gestor.agregar(r2); gestor.agregar(r3); Reserva r2mod = new Reserva(2, "Ana Maria", 3, 120); gestor.modificar(r2mod); gestor.listar();</pre>	<pre>Clave: 1 valor: Reserva{codigo=1, titular='Juan', noches=2, precioPorNoche=100.0, total=200.0} Clave: 2 valor: Reserva{codigo=2, titular='Ana Maria', noches=3, precioPorNoche=120.0, total=360.0} Clave: 3 valor: Reserva{codigo=3, titular='Luis', noches=1, precioPorNoche=150.0, total=150.0}</pre>	<pre>Clave: 1 valor: Reserva{codigo=1, titular='Juan', noches=2, precioPorNoche=100.0, total=200.0} Clave: 2 valor: Reserva{codigo=2, titular='Ana Maria', noches=3, precioPorNoche=120.0, total=360.0} Clave: 3 valor: Reserva{codigo=3, titular='Luis', noches=1, precioPorNoche=150.0, total=150.0}</pre>	✓

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 6,00/6,00.



Pregunta 27 | Correcta Se puntúa 6,00 sobre 6,00

Escribe un programa en Java que modele un sistema de cuentas bancarias.

Debes implementar una clase llamada **CuentaBancaria** y dos clases que hereden de ella: **CajaAhorro** y **PlazoFijo**.

La clase **CuentaBancaria** debe tener los siguientes atributos:

- **titular** de tipo **String**, que representa el nombre del titular de la cuenta.
- **saldo** de tipo **double**, que representa el saldo actual de la cuenta.

La clase **CuentaBancaria** debe tener:

- un constructor que permita inicializar todos sus atributos,
- los métodos getter y setter para cada atributo.

Además, la clase debe incluir los siguientes métodos:

- **calcularInteres(int meses)**: debe devolver el interés generado en una determinada cantidad de meses.
- **calcularSaldoFinal(int meses)**: debe devolver el saldo final luego de sumar el interés generado en esa cantidad de meses.

La clase **CajaAhorro** hereda de **CuentaBancaria** y debe sobrescribir estos métodos considerando un interés del **2% mensual**.

La clase **PlazoFijo** hereda de **CuentaBancaria** y debe sobrescribir estos métodos considerando un interés del **5% mensual**.

Para este ejercicio debes considerar que:

- el interés es **simple**,
- el interés se calcula con la fórmula: $\text{saldo} * \text{tasa} * \text{meses}$,
- el saldo final se calcula sumando el saldo inicial más el interés generado.

Importante:

- Debes respetar exactamente los nombres de las clases, atributos, constructores y métodos indicados.
- No debes desarrollar el método **main**, ya que nosotros nos encargaremos de realizar las pruebas.

Por ejemplo:

Prueba	Resultado
<pre>CajaAhorro cuenta1 = new CajaAhorro("Ana", 1000); System.out.println(cuenta1.getTitular()); System.out.println(cuenta1.getSaldo()); System.out.println(cuenta1.calcularInteres(3)); System.out.println(cuenta1.calcularSaldoFinal(3));</pre>	<pre>Ana 1000.0 60.0 1060.0</pre>
<pre>PlazoFijo cuenta2 = new PlazoFijo("Luis", 1000); System.out.println(cuenta2.getTitular()); System.out.println(cuenta2.getSaldo()); System.out.println(cuenta2.calcularInteres(3)); System.out.println(cuenta2.calcularSaldoFinal(3));</pre>	<pre>Luis 1000.0 150.0 1150.0</pre>

Respuesta: (sistema de penalización: 0, 0, 0, 10, 20, ... %)

Reiniciar respuesta

```

1 public class CuentaBancaria {
2     private String titular;
3     private double saldo;
4
5     public CuentaBancaria(String titular, double saldo) {
6         this.titular = titular;
7         this.saldo = saldo;
8     }
9
10    public String getTitular() {
11        return titular;
12    }
13
14    public void setTitular(String titular) {
15        this.titular = titular;
16    }
17 }
```




```

16     }
17
18     public double getSaldo() {
19         return saldo;
20     }
21
22     public void setSaldo(double saldo) {
23         this.saldo = saldo;
24     }
25
26     public double calcularInteres(int meses) {
27         return 0;
28     }
29
30     public double calcularSaldoFinal(int meses) {
31         return saldo + calcularInteres(meses);
32     }
33 }
34
35 // Clase CajaAhorro debe heredar de CuentaBancaria
36 class CajaAhorro extends CuentaBancaria {
37
38     public CajaAhorro(String titular, double saldo) {
39         super(titular, saldo);
40     }
41
42     @Override
43     public double calcularInteres(int meses) {
44         return getSaldo() * 0.02 * meses;
45     }
46
47     @Override
48     public double calcularSaldoFinal(int meses) {
49         return getSaldo() + calcularInteres(meses);
50     }
51 }
52

```

	Prueba	Esperado	Conseguido	
✓	CajaAhorro cuenta1 = new CajaAhorro("Ana", 1000); System.out.println(cuenta1.getTitular()); System.out.println(cuenta1.getSaldo()); System.out.println(cuenta1.calcularInteres(3)); System.out.println(cuenta1.calcularSaldoFinal(3));	Ana 1000.0 60.0 1060.0	Ana 1000.0 60.0 1060.0	✓
✓	PlazoFijo cuenta2 = new PlazoFijo("Luis", 1000); System.out.println(cuenta2.getTitular()); System.out.println(cuenta2.getSaldo()); System.out.println(cuenta2.calcularInteres(3)); System.out.println(cuenta2.calcularSaldoFinal(3));	Luis 1000.0 150.0 1150.0	Luis 1000.0 150.0 1150.0	✓
✓	CajaAhorro cuenta3 = new CajaAhorro("Maria", 500); cuenta3.setTitular("Maria Gomez"); cuenta3.setSaldo(2000); System.out.println(cuenta3.getTitular()); System.out.println(cuenta3.getSaldo()); System.out.println(cuenta3.calcularInteres(2)); System.out.println(cuenta3.calcularSaldoFinal(2));	Maria Gomez 2000.0 80.0 2080.0	Maria Gomez 2000.0 80.0 2080.0	✓



	Prueba	Esperado	Conseguido	
✓	<pre>PlazoFijo cuenta4 = new PlazoFijo("Pedro", 800); cuenta4.setTitular("Pedro Suarez"); cuenta4.setSaldo(1200); System.out.println(cuenta4.getTitular()); System.out.println(cuenta4.getSaldo()); System.out.println(cuenta4.calcularInteres(4)); System.out.println(cuenta4.calcularSaldoFinal(4));</pre>	Pedro Suarez 1200.0 240.0 1440.0	Pedro Suarez 1200.0 240.0 1440.0	✓
✓	<pre>CajaAhorro cuenta5 = new CajaAhorro("Lucia", 1500); System.out.println(cuenta5.calcularInteres(1)); System.out.println(cuenta5.calcularSaldoFinal(1));</pre>	30.0 1530.0	30.0 1530.0	✓
✓	<pre>PlazoFijo cuenta6 = new PlazoFijo("Jorge", 1500); System.out.println(cuenta6.calcularInteres(1)); System.out.println(cuenta6.calcularSaldoFinal(1));</pre>	75.0 1575.0	75.0 1575.0	✓
✓	<pre>CajaAhorro cuenta7 = new CajaAhorro("Sofia", 0); System.out.println(cuenta7.getTitular()); System.out.println(cuenta7.getSaldo()); System.out.println(cuenta7.calcularInteres(6)); System.out.println(cuenta7.calcularSaldoFinal(6));</pre>	Sofia 0.0 0.0 0.0	Sofia 0.0 0.0 0.0	✓
✓	<pre>PlazoFijo cuenta8 = new PlazoFijo("Carlos", 2500); System.out.println(cuenta8.getTitular()); System.out.println(cuenta8.getSaldo()); System.out.println(cuenta8.calcularInteres(5)); System.out.println(cuenta8.calcularSaldoFinal(5));</pre>	Carlos 2500.0 625.0 3125.0	Carlos 2500.0 625.0 3125.0	✓
✓	<pre>CuentaBancaria cuenta9 = new CajaAhorro("Valeria", 3000); System.out.println(cuenta9.getTitular()); System.out.println(cuenta9.getSaldo()); System.out.println(cuenta9.calcularInteres(2)); System.out.println(cuenta9.calcularSaldoFinal(2));</pre>	Valeria 3000.0 120.0 3120.0	Valeria 3000.0 120.0 3120.0	✓
✓	<pre>CuentaBancaria cuenta10 = new PlazoFijo("Diego", 3000); System.out.println(cuenta10.getTitular()); System.out.println(cuenta10.getSaldo()); System.out.println(cuenta10.calcularInteres(9)); System.out.println(cuenta10.calcularSaldoFinal(9));</pre>	Diego 3000.0 1350.0 4350.0	Diego 3000.0 1350.0 4350.0	✓

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 6,00/6,00.



Pregunta 28 | Finalizado Se puntúa 1,34 sobre 1,34

Dado el siguiente código en lenguaje Java:

```
public class Booleano {  
    public static void main(String[] args) {  
        boolean a = true;  
        boolean b = false;  
        boolean c = (a && !b) ? !(a || b) : (a && b);  
  
        System.out.println(!c);  
    }  
}
```

¿ Al ejecutar este programa cual es el contenido de la variable c al final del mismo ?

- ☐ a. Da error de compilación.
- ☒ b. false
- ☐ c. true
- ☐ d. No se puede determinar.

Pregunta 29 | Finalizado Se puntúa 1,33 sobre 1,33

Si se tiene una clase Auto con atributos marca y modelo, y luego se crea:

Auto a1 = new Auto();

¿Qué representa a1?

- ☐ a. Una parte del constructor
- ☐ b. Un atributo
- ☐ c. Una clase
- ☒ d. Un objeto o instancia de Auto

Pregunta 30 | Finalizado Se puntúa 1,33 sobre 1,33

Dado el siguiente método:

public static void mostrarMensaje()

¿cómo se clasifica?

- ☒ a. Como un procedimiento, porque no devuelve valor
- ☐ b. Como una función, porque es public
- ☐ c. Como un procedimiento, porque es static
- ☐ d. Como una función, porque no recibe parámetros



Pregunta 31 | Correcta Se puntúa 6,00 sobre 6,00

Escribe un programa en Java que modele una clase llamada **Libro**.

La clase debe tener los siguientes atributos:

- **titulo** de tipo **String**, que representa el título del libro.
- **autor** de tipo **String**, que representa el autor del libro.
- **cantidadPaginas** de tipo **int**, que representa la cantidad de páginas del libro.

La clase debe implementar los siguientes constructores:

1. Un constructor que reciba **titulo**, **autor** y **cantidadPaginas**.
2. Un constructor que reciba únicamente **titulo** y **autor**. En este caso, **cantidadPaginas** debe quedar con valor **0**.
3. Un constructor sin parámetros. En este caso:
 - **titulo** debe quedar con **""**
 - **autor** debe quedar con **""**
 - **cantidadPaginas** debe quedar con **0**

Además, la clase debe tener los métodos **getter** y **setter** para cada uno de sus atributos.

La clase también debe implementar los siguientes métodos:

- **mostrarInformacion()**: debe devolver un **String** con la información del libro en un formato legible.
- **esLargo()**: debe devolver **true** si el libro tiene **230 páginas o más**, y **false** en caso contrario.

Importante:

- No debes desarrollar el método **main**, ya que nosotros nos encargaremos de realizar las pruebas.
- Debes respetar exactamente los nombres indicados para la clase, los atributos, los constructores y los métodos.

Resumen de lo esperado en la clase:

- Clase: **Libro**
- Atributos:
 - **titulo**
 - **autor**
 - **cantidadPaginas**
- Constructores:
 - **Libro(String titulo, String autor, int cantidadPaginas)**
 - **Libro(String titulo, String autor)**
 - **Libro()**
- Métodos:
 - getters y setters de todos los atributos
 - **String mostrarInformacion()**
 - **boolean esLargo()**

Por ejemplo:

Prueba	Resultado
<pre>Libro libro1 = new Libro("El Principito", "Antoine de Saint-Exupery", 96); System.out.println(libro1.getTitulo()); System.out.println(libro1.getAutor()); System.out.println(libro1.getCantidadPaginas()); System.out.println(libro1.mostrarInformacion()); System.out.println(libro1.esLargo());</pre>	<pre>El Principito Antoine de Saint-Exupery 96 Titulo: El Principito, Autor: Antoine de Saint-Exupery, Cantidad de paginas: 96 false</pre>



Prueba	Resultado
<pre> Libro libro2 = new Libro("Martin Fierro", "Jose Hernandez"); System.out.println(libro2.getTitulo()); System.out.println(libro2.getAutor()); System.out.println(libro2.getCantidadPaginas()); System.out.println(libro2.mostrarInformacion()); System.out.println(libro2.esLargo()); </pre>	<pre> Martin Fierro Jose Hernandez 0 Titulo: Martin Fierro, Autor: Jose Hernandez, Cantidad de paginas: 0 false </pre>

Respuesta: (sistema de penalización: 0, 0, 0, 10, 20, 30, ... %)

Reiniciar respuesta

```

1 public class Libro {
2     private String titulo;
3     private String autor;
4     private int cantidadPaginas;
5
6     public Libro(String titulo, String autor, int cantidadPaginas) {
7         this.titulo = titulo;
8         this.autor = autor;
9         this.cantidadPaginas = cantidadPaginas;
10    }
11
12    public Libro(String titulo, String autor) {
13        this.titulo = titulo;
14        this.autor = autor;
15        this.cantidadPaginas = 0;
16    }
17
18    public Libro() {
19        this.titulo = "";
20        this.autor = "";
21        this.cantidadPaginas = 0;
22    }
23
24    public String getTitulo() {
25        return titulo;
26    }
27
28    public void setTitulo(String titulo) {
29        this.titulo = titulo;
30    }
31
32    public String getAutor() {
33        return autor;
34    }
35
36    public void setAutor(String autor) {
37        this.autor = autor;
38    }
39
40    public int getCantidadPaginas() {
41        return cantidadPaginas;
42    }
43
44    public void setCantidadPaginas(int cantidadPaginas) {
45        this.cantidadPaginas = cantidadPaginas;
46    }
47
48    public String mostrarInformacion() {
49        return "Titulo: " + titulo +
50            ", Autor: " + autor +
51            ", Cantidad de paginas: " + cantidadPaginas;
52    }

```



	Prueba	Esperado	Conseguido	
✓	<pre> Libro libro1 = new Libro("El Principito", "Antoine de Saint-Exupery", 96); System.out.println(libro1.getTitulo()); System.out.println(libro1.getAutor()); System.out.println(libro1.getCantidadPaginas()); System.out.println(libro1.mostrarInformacion()); System.out.println(libro1.esLargo()); </pre>	<pre> El Principito Antoine de Saint-Exupery 96 Titulo: El Principito, Autor: Antoine de Saint-Exupery, Cantidad de paginas: 96 false </pre>	<pre> El Principito Antoine de Saint-Exupery 96 Titulo: El Principito, Autor: Antoine de Saint-Exupery, Cantidad de paginas: 96 false </pre>	✓
✓	<pre> Libro libro2 = new Libro("Martin Fierro", "Jose Hernandez"); System.out.println(libro2.getTitulo()); System.out.println(libro2.getAutor()); System.out.println(libro2.getCantidadPaginas()); System.out.println(libro2.mostrarInformacion()); System.out.println(libro2.esLargo()); </pre>	<pre> Martin Fierro Jose Hernandez 0 Titulo: Martin Fierro, Autor: Jose Hernandez, Cantidad de paginas: 0 false </pre>	<pre> Martin Fierro Jose Hernandez 0 Titulo: Martin Fierro, Autor: Jose Hernandez, Cantidad de paginas: 0 false </pre>	✓
✓	<pre> Libro libro3 = new Libro(); System.out.println(libro3.getTitulo()); System.out.println(libro3.getAutor()); System.out.println(libro3.getCantidadPaginas()); System.out.println(libro3.mostrarInformacion()); System.out.println(libro3.esLargo()); </pre>	<pre> 0 Titulo: , Autor: , Cantidad de paginas: 0 false </pre>	<pre> 0 Titulo: , Autor: , Cantidad de paginas: 0 false </pre>	✓
✓	<pre> Libro libro4 = new Libro(); libro4.setTitulo("Don Quijote de la Mancha"); libro4.setAutor("Miguel de Cervantes"); libro4.setCantidadPaginas(863); System.out.println(libro4.getTitulo()); System.out.println(libro4.getAutor()); System.out.println(libro4.getCantidadPaginas()); System.out.println(libro4.mostrarInformacion()); System.out.println(libro4.esLargo()); </pre>	<pre> Don Quijote de la Mancha Miguel de Cervantes 863 Titulo: Don Quijote de la Mancha, Autor: Miguel de Cervantes, Cantidad de paginas: 863 true </pre>	<pre> Don Quijote de la Mancha Miguel de Cervantes 863 Titulo: Don Quijote de la Mancha, Autor: Miguel de Cervantes, Cantidad de paginas: 863 true </pre>	✓
✓	<pre> Libro libro5 = new Libro("Libro de Prueba", "Autor de Prueba", 230); System.out.println(libro5.getCantidadPaginas()); System.out.println(libro5.esLargo()); </pre>	<pre> 230 true </pre>	<pre> 230 true </pre>	✓
✓	<pre> Libro libro6 = new Libro("Otro Libro", "Otro Autor", 229); System.out.println(libro6.getCantidadPaginas()); System.out.println(libro6.esLargo()); </pre>	<pre> 229 false </pre>	<pre> 229 false </pre>	✓
✓	<pre> Libro libro7 = new Libro("Cien Años de Soledad", "Gabriel Garcia Marquez"); libro7.setCantidadPaginas(471); System.out.println(libro7.mostrarInformacion()); System.out.println(libro7.esLargo()); </pre>	<pre> Titulo: Cien Años de Soledad, Autor: Gabriel Garcia Marquez, Cantidad de paginas: 471 true </pre>	<pre> Titulo: Cien Años de Soledad, Autor: Gabriel Garcia Marquez, Cantidad de paginas: 471 true </pre>	✓



	Prueba	Esperado	Conseguido	
✓	<pre>Libro libro8 = new Libro("A", "B", 10); libro8.setTitulo("Harry Potter y la piedra filosofal"); libro8.setAutor("J.K. Rowling"); libro8.setCantidadPaginas(223); System.out.println(libro8.getTitulo()); System.out.println(libro8.getAutor()); System.out.println(libro8.getCantidadPaginas()); System.out.println(libro8.mostrarInformacion()); System.out.println(libro8.esLargo());</pre>	<pre>Harry Potter y la piedra filosofal J.K. Rowling 223 Titulo: Harry Potter y la piedra filosofal, Autor: J.K. Rowling, Cantidad de paginas: 223 false</pre>	<pre>Harry Potter y la piedra filosofal J.K. Rowling 223 Titulo: Harry Potter y la piedra filosofal, Autor: J.K. Rowling, Cantidad de paginas: 223 false</pre>	✓

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 6,00/6,00.

Pregunta 32 | Finalizado Se puntúa 1,33 sobre 1,33

Supongamos que tenemos la clase Animal, y luego tenemos la clase Perro que hereda de la clase Animal.

Y la clase Perro redefine el método llamado hacerSonido() :

Y tenemos el siguiente código :

```
Animal a = new Perro();
a.hacerSonido();
```

¿Qué concepto se está aplicando?

- ☐ a. Encapsulamiento
- ☐ b. Abstracción
- ☒ c. Polimorfismo
- ☐ d. Sobrecarga



Pregunta 33 | Finalizado Se puntúa 1,34 sobre 1,34

Dado el siguiente código en Java, donde vemos un caso de Herencia, donde las clases Perro y Gato heredan de la clase Animal.

```
public class PruebaHerencia {  
    public static void main(String[] args) {  
        Animal animal = new Animal();  
        Animal perro = new Perro();  
        Animal gato = new Gato();  
  
        System.out.print((animal instanceof Animal) + " - ");  
        System.out.print((animal instanceof Perro) + " - ");  
        System.out.print((perro instanceof Animal) + " - ");  
        System.out.print((perro instanceof Perro) + " - ");  
        System.out.print((gato instanceof Perro) + " - ");  
        System.out.print((gato instanceof Animal) + " - ");  
    }  
}
```

¿ Qué se muestra en la consola al ejecutar este programa ?

- ☐ a. true - true - true - false - false - true -
- ☒ b. true - false - true - true - false - true -
- ☐ c. true - false - false - true - false - true -
- ☐ d. false - false - true - true - false - true -



Pregunta 34 | Correcta Se puntúa 6,00 sobre 6,00

Tenemos un procedimiento que recibe un parámetro de tipo `String`.

Debes completar el código del procedimiento para controlar las posibles excepciones que puedan producirse al ejecutar las instrucciones dadas.

La firma del procedimiento es la siguiente:

```
public void prueba(String dato)
```

Dentro del procedimiento se intenta trabajar con el `String` recibido realizando las siguientes operaciones:

- acceder a un carácter en una posición determinada,
- obtener una subcadena a partir de una parte del texto.

A partir del siguiente código base, debes agregar el control de excepciones necesario para evitar que el programa finalice de forma abrupta:

```
public void prueba(String dato){  
    System.out.println(dato.charAt(3));  
    System.out.println(dato.substring(0, 5));  
    System.out.println("Proceso correcto");  
}
```

Debes controlar las siguientes excepciones:

- Si se produce una excepción por trabajar con una referencia `null`, debes mostrar exactamente el mensaje:
`Error de null`
- Si se produce una excepción porque el `String` no tiene la longitud suficiente para realizar alguna de las operaciones, debes mostrar exactamente el mensaje:
`Error de string`

Si no ocurre ninguna excepción, deben mostrarse los resultados de las operaciones y luego el mensaje:

`Proceso correcto`

Importante:

- No debes incluir el método `main`, ya que nosotros nos encargaremos de realizar las pruebas.
- No debes probar el método con llamadas manuales.
- Solo debes completar la lógica del procedimiento.
- Debes respetar exactamente los mensajes indicados para cada caso.

Por ejemplo:

Prueba	Resultado
<code>prueba("Programacion");</code>	<code>g</code> <code>Progr</code> <code>Proceso correcto</code>
<code>prueba("Hola");</code>	<code>a</code> <code>Error de string</code>
<code>prueba(null);</code>	<code>Error de null</code>

Respuesta: (sistema de penalización: 0, 0, 0, 10, 20, 30, ... %)

Reiniciar respuesta

```
1 public void prueba(String dato) {  
2     try {  
3         System.out.println(dato.charAt(3));  
4         System.out.println(dato.substring(0, 5));  
5         System.out.println("Proceso correcto");  
6     } catch (NullPointerException e) {  
7         System.out.println("Error de null");  
8     } catch (StringIndexOutOfBoundsException e) {  
9         System.out.println("Error de string");  
10    }
```



11 | }

11

	Prueba	Esperado	Conseguido	
✓	prueba("Programacion");	g Progr Proceso correcto	g Progr Proceso correcto	✓
✓	prueba("Computadora");	p Compu Proceso correcto	p Compu Proceso correcto	✓
✓	prueba("Hola");	a Error de string	a Error de string	✓
✓	prueba("Sol");	Error de string	Error de string	✓
✓	prueba(null);	Error de null	Error de null	✓
✓	prueba("");	Error de string	Error de string	✓
✓	prueba("Banana");	a Banan Proceso correcto	a Banan Proceso correcto	✓
✓	prueba("Java");	a Error de string	a Error de string	✓

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 6,00/6,00.



Pregunta 35 | Correcta Se puntúa 6,00 sobre 6,00

Objetivo: Recorrer un array, filtrar valores según una condición (números pares) y calcular un promedio.

Se cuenta con:

- Un método a implementar llamado:
`calcularPromedioPares(int[] array)`
- Un array de números enteros que será utilizado como entrada.

✓ **Tu tarea:** Implementar el método `calcularPromedioPares` para que:

- Recorra el array.
- Identifique cuáles son los números pares.
- Sume únicamente los números pares.
- Cuente cuántos números pares hay.
- Calcule y devuelva el promedio como un número decimal.

Importante:

- El método debe ser `static`.
- No se debe implementar el método `main`.
- No se deben pedir datos por teclado.
- Si el array no contiene números pares, el método debe devolver `0`.

Por ejemplo:

Prueba	Resultado
<pre>int[] numeros1 = {2, 4, 6, 8}; System.out.println(calcularPromedioPares(numeros1));</pre>	5.0
<pre>int[] numeros2 = {1, 3, 5, 7}; System.out.println(calcularPromedioPares(numeros2));</pre>	0.0

Respuesta: (sistema de penalización: 0, 0, 0, 10, 20, 30, ... %)

Reiniciar respuesta

```

1 public static double calcularPromedioPares(int[] array) {
2     int suma = 0;
3     int contador = 0;
4
5     for (int i = 0; i < array.length; i++) {
6         if (array[i] % 2 == 0) {
7             suma += array[i];
8             contador++;
9         }
10    }
11
12    if (contador == 0) {
13        return 0;
14    }
15
16    return (double) suma / contador;
17 }
```

	Prueba	Esperado	Conseguido	
✓	<pre>int[] numeros1 = {2, 4, 6, 8}; System.out.println(calcularPromedioPares(numeros1));</pre>	5.0	5.0	✓



	Prueba	Esperado	Conseguido	
✓	int[] numeros2 = {1, 3, 5, 7}; System.out.println(calcularPromedioPares(numeros2));	0.0	0.0	✓
✓	int[] numeros3 = {1, 2, 3, 4, 5, 6}; System.out.println(calcularPromedioPares(numeros3));	4.0	4.0	✓
✓	int[] numeros4 = {10}; System.out.println(calcularPromedioPares(numeros4));	10.0	10.0	✓
✓	int[] numeros5 = {9}; System.out.println(calcularPromedioPares(numeros5));	0.0	0.0	✓
✓	int[] numeros6 = {-2, -4, 6, 8}; System.out.println(calcularPromedioPares(numeros6));	2.0	2.0	✓
✓	int[] numeros7 = {0, 2, 4, 6}; System.out.println(calcularPromedioPares(numeros7));	3.0	3.0	✓
✓	int[] numeros8 = {}; System.out.println(calcularPromedioPares(numeros8));	0.0	0.0	✓
✓	int[] numeros9 = {0,0,0,10}; System.out.println(calcularPromedioPares(numeros9));	2.5	2.5	✓
✓	int[] numeros11 = {3, 8, 11, 14, 7, 20, 9, 6, 13, 18, 5, 10, 21, 4, 12}; System.out.println(calcularPromedioPares(numeros11));	11.5	11.5	✓
✓	int[] numeros12 = {5, 12, 7, 18, 9, 24, 11, 6, 13, 20, 15, 10, 17, 8, 21, 14}; System.out.println(calcularPromedioPares(numeros12));	14.0	14.0	✓

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 6,00/6,00.



Pregunta 36 | Correcta Se puntúa 6,00 sobre 6,00

Completar la función validar letras, se debe pasar como parámetro un String que contenga letras, estas letras deben de ser las letras vocales, y el largo del String es valido si el largo es entre 3 y 6 letras. Se debe tomar como letras vocales tanto las minúsculas como las mayúsculas.

Se deben cumplir ambas condiciones:

La firma de la función es la siguiente:

```
public boolean validarVocal(String palabra)
```

- No debes incluir el método main, solo completar la lógica del método.
- No debes probar el método con llamadas, de eso se encarga CodeRunner, solo completar la lógica del método.

Por ejemplo:

Prueba	Resultado
String palabra="Aei"; System.out.println(validarVocal(palabra));	true
String palabra="aeijk"; System.out.println(validarVocal(palabra));	false

Respuesta: (sistema de penalización: 0, 0, 0, 10, 20, 30, ... %)

Reiniciar respuesta

```
1 public boolean validarVocal(String palabra) {
2     if (palabra.length() < 3 || palabra.length() > 6) {
3         return false;
4     }
5
6     for (int i = 0; i < palabra.length(); i++) {
7         char letra = palabra.charAt(i);
8
9         if (letra != 'a' && letra != 'e' && letra != 'i' && letra != 'o' && letra != 'u' &&
10            letra != 'A' && letra != 'E' && letra != 'I' && letra != 'O' && letra != 'U') {
11             return false;
12         }
13     }
14
15     return true;
16 }
```

	Prueba	Esperado	Conseguido	
✓	String palabra="Aei"; System.out.println(validarVocal(palabra));	true	true	✓
✓	String palabra="aeijk"; System.out.println(validarVocal(palabra));	false	false	✓
✓	String palabra="ae"; System.out.println(validarVocal(palabra));	false	false	✓
✓	String palabra="AEIju"; System.out.println(validarVocal(palabra));	false	false	✓
✓	String palabra="aeiAEIOU"; System.out.println(validarVocal(palabra));	false	false	✓



	Prueba	Esperado	Conseguido	
✓	String palabra="aeiAEI"; System.out.println(validarVocal(palabra));	true	true	✓
✓	String palabra="aei@aI"; System.out.println(validarVocal(palabra));	false	false	✓
✓	String palabra="AáauI"; System.out.println(validarVocal(palabra));	false	false	✓

Todas las pruebas superadas. ✓

Correcta

Puntos para este envío: 6,00/6,00.

Pregunta 37 | Finalizado Se puntúa 1,33 sobre 1,33

Dada la siguiente firma :

public static boolean esPar(int numero)

¿ Qué representa boolean?

- ☐ a. El modificador de acceso
- ☐ b. El tipo del parámetro
- ☐ c. El nombre del método
- ☒ d. El tipo de retorno



Pregunta 38 | Finalizado Se puntúa 1,34 sobre 1,34

Dado el siguiente código en lenguaje Java:

```
public class ProbarSwitch {  
    public static void main(String[] args) {  
        for (int i = 4; i > 0; i--) {  
            switch (i - 2) {  
                case 0:  
                    System.out.print("A-");  
                    break;  
                case 1:  
                    System.out.print("B-");  
                case 2:  
                    System.out.print("C-");  
                    break;  
                default:  
                    System.out.print("D-");  
            }  
        }  
    }  
}
```

¿ Qué se imprime en la consola al ejecutar este programa ?

- ☐ a. C-B-A-D-C-
- ☒ b. C-B-C-A-D-
- ☐ c. C-B-C-A-
- ☐ d. A-C-B-A-

Pregunta 39 | Finalizado Se puntúa 1,34 sobre 1,34

Dado el siguiente código en lenguaje Java:

```
public class PruebaLista {  
    public static void main(String[] args) {  
        String[] nombres = {"Ana", "Luis", "Marta"};  
        List<String> lista = Arrays.asList(nombres);  
  
        lista.add("Pedro");  
  
        for (String n : lista) {  
            System.out.print(n + " ");  
        }  
    }  
}
```

¿ Qué se muestra en la consola al ejecutar este programa ?

- ☒ a. Se produce una excepción en tiempo de ejecución
- ☐ b. Se imprime: Ana Luis Marta
- ☐ c. Se produce un error de compilación
- ☐ d. Se imprime: Ana Luis Marta Pedro



Pregunta 40 | Finalizado Se puntúa 1,33 sobre 1,33

Dada la siguiente estructura de un programa en Java:

```
// Bloque 1
try {
    // Bloque 2
} catch (NullPointerException ex) {
    // Bloque 3
} catch (Exception e) {
    // Bloque 4
} finally {
    // Bloque 5
}
// Bloque 6
```

Si no ocurre ninguna excepción durante la ejecución, ¿ cuál es el flujo correcto ?

- ☐ a. Bloque 2 -> Bloque 5 -> Bloque 6
- ☐ b. Bloque 1 -> Bloque 2 -> Bloque 6
- ☐ c. Bloque1 -> Bloque 2 -> Bloque 4 -> Bloque 5 -> Bloque 6
- ☒ d. Bloque 1 -> Bloque 2 -> Bloque 5 -> Bloque 6

[Regresar a Curso](#)

